

JUC框架

一、前言

1.1 参考文章

- 原子类相关: https://blog.csdn.net/zy_dreamer/article/details/132564973
- 原子类相关: <https://juejin.cn/post/6874705527473963015>
- 内存可见性问题: <https://www.cnblogs.com/frydsh/p/5720658.html>

1.2 说明

JUC指的是 `java.util.concurrent` 包下的所有提供的工具类的简称。这是一个处理线程的工具包，JDK1.5开始出现的。

1.3 分类

JUC可以分为如下几类：

- Lock框架和Tools类
- Collections并发集合
- Atomic：原子类
- Executors：线程池

1.4 实现线程安全方式

- 互斥同步：synchronized和ReentrantLock
- 非阻塞同步：CAS（Compare-And-Swap，对比交换，基于硬件平台汇编指令实现，是一条CPU的原子指令，JVM只是封装了汇编的调用）、AtomicXXX原子类（Java原子类底层底层通过使用Unsafe实现CAS，使用volatile保证线程的可见性，使用CAS保证数据更新的原子性）
- 无同步方式：栈封闭、ThreadLocal、可重入代码

1.5 前置知识点

synchronized和volatile以及final（△）。

二、CAS、Unsafe和原子类

2.1 说明

CAS：Compare-And-Swap，对比交换，基于硬件平台汇编指令实现，是一条CPU的原子指令，让CPU进行比较两个值是否相等，然后原子地更新某个位置的值。JVM只是封装了汇编的调用，那些原子类就是使用了这些封装后的接口。其相对于synchronized悲观锁来说，其是一个乐观锁（因此CAS解决并发问题通常情况下性能会更优）。其实现大概就是CAS操作需要输入两个数值，一个旧值和一个新值，在操作过程中先比较旧值有没有发生变化，如果没有发生变化，就替换成新值，否则不进行替换。CAS操作过程中，会有如下三个问题比较突出：

- **ABA问题**
循环时间长开销大（自旋CAS长时间不成功）
- 只能保证一个共享变量的原子操作（JDK1.5开始，提供了AtomicReference类来保证引用对象之间的原子性）

Unsafe: Java中的原子类都是通过Unsafe类提供的CAS操作实现的，其是位于sun.misc包下的一个类，其提供了一些用于执行低级别、不安全操作的方法。其中有很多public方法，但都是本地方法，并且对于Unsafe类的使用都是受限制的，只有授信的代码才能获得该类的实例（JDK库里面的类都是可以随意使用的），从反编译的代码来看，其内部使用自旋的方式进行CAS更新（while循环进行CAS更新，如果更新失败，则循环再次重试）。其提供的CAS操作其实只有如下三种：

```
public final native boolean compareAndSwapObject(Object var1, long var2, Object var4, Object var5);

public final native boolean compareAndSwapInt(Object var1, long var2, int var4, int var5);

public final native boolean compareAndSwapLong(Object var1, long var2, long var4, long var6);
```

原子类: 在多线程下，提供CAS操作的类。

2.2 原子类

0.说明

Java中的原子变量使我们能够轻松地对类的引用或字段进行线程安全的操作，而不需要添加监视器或互斥等并发原语。它们被定义在 java.util.concurrent.atomic 包下。

1. 原子更新基本类型

- AtomicInteger

使用volatile修饰维护的变量value，保证线程的可见性，当多线程并发时，一个线程修改数据，可以保证其他线程立马看到修改后的值。】

通过Unsafe类提供的CAS操作，保证数据更新的原子性

△获取当前值get方法（为什么要用final修饰呢1.保证可见性，这个不是volatile就保证了吗 2.性能优化）

```
public class AtomicInteger extends Number implements java.io.Serializable {
    private static final long serialVersionUID = 6214790243416807050L;

    // setup to use Unsafe.compareAndSwapInt for updates
    // 设置为使用Unsafe.compareAndSwapInt进行更新（使用unsafe提供的CAS操作进行原子更新）
    private static final Unsafe unsafe = Unsafe.getUnsafe();

    // 存储 value 字段在 AtomicInteger 对象内存中的偏移量。这个偏移量是一个整数值，表示从对象的起始地址到 value 字段的字节数
    // 即这个对象在内存中的地址，方柏霓后续通过内存地址直接进行操作value值
    private static final long valueOffset;

    static {
        try {
```

```

        // 静态方法用于获取value字段相对当前对象的起始地址的偏移量
        valueOffset = unsafe.objectFieldOffset
            (AtomicInteger.class.getDeclaredField("value"));
    } catch (Exception ex) { throw new Error(ex); }
}

// 维护的value值
private volatile int value;

/**
 * Creates a new AtomicInteger with the given initial value.
 * 用给定的初始值创建一个新的AtomicInteger
 *
 * @param initialValue the initial value
 */
public AtomicInteger(int initialValue) {
    value = initialValue;
}

/**
 * Creates a new AtomicInteger with initial value {@code 0}.
 * 创建一个新的AtomicInteger，初始值为0（因为是int类型）
 */
public AtomicInteger() {
}

/**
 * Gets the current value.
 * 原子性地获取当前值（为什么要用final修饰呢1.保证可见性 2.性能优化）
 * 各个线程对变量value都是可见的，因为value变量被volatile修饰
 * @return the current value
 */
public final int get() {
    return value;
}

/**
 * Sets to the given value.
 * 原子性地设置为给定的值。
 * 各个线程对变量value都是可见的，因为value变量被volatile修饰
 * @param newValue the new value
 */
public final void set(int newValue) {
    value = newValue;
}

/**
 * Eventually sets to the given value.
 * 最终设置为给定值，取消内存屏障（可能导致其他线程在之后的一小段时间内还是可以读到旧的值）
 * 见详情1
 * @param newValue the new value
 * @since 1.6
 */
public final void lazySet(int newValue) {
    unsafe.putOrderedInt(this, valueOffset, newValue);
}

```

```

/**
 * Atomically sets to the given value and returns the old value.
 * 以原子方式设置为给定值并返回旧值
 * 见详情2
 * @param newValue the new value
 * @return the previous value
 */
public final int getAndSet(int newValue) {
    // 调用unsafe的getAndSetInt方法
    // 参数: 1.当前实例 2.value实例变量的偏移量 3.当前value要设置的新值
    return unsafe.getAndSetInt(this, valueOffset, newValue);
}

/**
 * Atomically sets the value to the given updated value
 * if the current value {@code == } the expected value.
 *
 * 见详情3
 * @param expect the expected value
 * @param update the new value
 * @return {@code true} if successful. False return indicates that
 * the actual value was not equal to the expected value.
 */
public final boolean compareAndSet(int expect, int update) {
    return unsafe.compareAndSwapInt(this, valueOffset, expect, update);
}

/**
 * Atomically sets the value to the given updated value
 * if the current value {@code == } the expected value.
 *
 * <p><a href="package-summary.html#weakCompareAndSet">May fail
 * spuriously and does not provide ordering guarantees</a>, so is
 * only rarely an appropriate alternative to {@code compareAndSet}.
 *
 * 见详情4
 * @param expect the expected value
 * @param update the new value
 * @return {@code true} if successful
 */
public final boolean weakCompareAndSet(int expect, int update) {
    return unsafe.compareAndSwapInt(this, valueOffset, expect, update);
}

/**
 * Atomically increments by one the current value.
 * 将当前值自动加1。类似于 i = i ++
 * 见详情5
 * @return the previous value
 */
public final int getAndIncrement() {
    return unsafe.getAndAddInt(this, valueOffset, 1);
}

/**

```

```

    * Atomically decrements by one the current value.
    * 将当前值自动减1。类似于 i = i --
    *
    * @return the previous value
    */
    public final int getAndDecrement() {
        return unsafe.getAndAddInt(this, valueOffset, -1);
    }

    /**
     * Atomically adds the given value to the current value.
     *
     * @param delta the value to add
     * @return the previous value
     */
    public final int getAndAdd(int delta) {
        return unsafe.getAndAddInt(this, valueOffset, delta);
    }

    /**
     * Atomically increments by one the current value.
     * 将当前值自动加1。类似于 i = ++ i
     * @return the updated value
     */
    public final int incrementAndGet() {
        return unsafe.getAndAddInt(this, valueOffset, 1) + 1;
    }

    /**
     * Atomically decrements by one the current value.
     * 将当前值自动减1。类似于 i = -- i
     * @return the updated value
     */
    public final int decrementAndGet() {
        return unsafe.getAndAddInt(this, valueOffset, -1) - 1;
    }

    /**
     * Atomically adds the given value to the current value.
     *
     * @param delta the value to add
     * @return the updated value
     */
    public final int addAndGet(int delta) {
        return unsafe.getAndAddInt(this, valueOffset, delta) + delta;
    }

    /**
     * Atomically updates the current value with the results of
     * applying the given function, returning the previous value. The
     * function should be side-effect-free, since it may be re-applied
     * when attempted updates fail due to contention among threads.
     *
     * @param updateFunction a side-effect-free function
     * @return the previous value
     * @since 1.8

```

```

    */
    public final int getAndUpdate(IntUnaryOperator updateFunction) {
        int prev, next;
        do {
            prev = get();
            next = updateFunction.applyAsInt(prev);
        } while (!compareAndSet(prev, next));
        return prev;
    }

    /**
     * Atomically updates the current value with the results of
     * applying the given function, returning the updated value. The
     * function should be side-effect-free, since it may be re-applied
     * when attempted updates fail due to contention among threads.
     *
     * @param updateFunction a side-effect-free function
     * @return the updated value
     * @since 1.8
     */
    public final int updateAndGet(IntUnaryOperator updateFunction) {
        int prev, next;
        do {
            prev = get();
            next = updateFunction.applyAsInt(prev);
        } while (!compareAndSet(prev, next));
        return next;
    }

    /**
     * Atomically updates the current value with the results of
     * applying the given function to the current and given values,
     * returning the previous value. The function should be
     * side-effect-free, since it may be re-applied when attempted
     * updates fail due to contention among threads. The function
     * is applied with the current value as its first argument,
     * and the given update as the second argument.
     *
     * @param x the update value
     * @param accumulatorFunction a side-effect-free function of two arguments
     * @return the previous value
     * @since 1.8
     */
    public final int getAndAccumulate(int x,
                                     IntBinaryOperator accumulatorFunction) {
        int prev, next;
        do {
            prev = get();
            next = accumulatorFunction.applyAsInt(prev, x);
        } while (!compareAndSet(prev, next));
        return prev;
    }

    /**
     * Atomically updates the current value with the results of
     * applying the given function to the current and given values,

```

```

    * returning the updated value. The function should be
    * side-effect-free, since it may be re-applied when attempted
    * updates fail due to contention among threads. The function
    * is applied with the current value as its first argument,
    * and the given update as the second argument.
    *
    * @param x the update value
    * @param accumulatorFunction a side-effect-free function of two arguments
    * @return the updated value
    * @since 1.8
    */
    public final int accumulateAndGet(int x,
                                      IntBinaryOperator accumulatorFunction) {
        int prev, next;
        do {
            prev = get();
            next = accumulatorFunction.applyAsInt(prev, x);
        } while (!compareAndSet(prev, next));
        return next;
    }

    /**
     * Returns the String representation of the current value.
     * 返回当前值的字符串表示形式。
     * @return the String representation of the current value
     */
    public String toString() {
        return Integer.toString(get());
    }

    /**
     * Returns the value of this {@code AtomicInteger} as an {@code int}.
     * 返回AtomicInteger作为int的值。
     */
    public int intValue() {
        return get();
    }

    /**
     * Returns the value of this {@code AtomicInteger} as a {@code long}
     * after a widening primitive conversion.
     * 在扩展原语转换后, 返回AtomicInteger作为long的值。
     * @jls 5.1.2 Widening Primitive Conversions 扩展原始转换
     */
    public long longValue() {
        return (long) get();
    }

    /**
     * Returns the value of this {@code AtomicInteger} as a {@code float}
     * after a widening primitive conversion.
     * 在扩展原语转换后, 返回AtomicInteger作为float的值。
     * @jls 5.1.2 Widening Primitive Conversions 扩展原始转换
     */
    public float floatValue() {
        return (float) get();
    }

```

```

    }

    /**
     * Returns the value of this {@code AtomicInteger} as a {@code double}
     * after a widening primitive conversion.
     * 在扩展原语转换后，返回AtomicInteger作为double的值。
     * @jls 5.1.2 Widening Primitive Conversions 扩展原始转换
     */
    public double doubleValue() {
        return (double)get();
    }
}

```

详情1 lazySet方法

参考文章: <https://blog.csdn.net/bluetjs/article/details/52423867>

```

public final void lazySet(int newValue) {
    unsafe.putOrderedInt(this, valueOffset, newValue);
}

```

其中调用了unsafe类的putOrderedInt方法，其是一个本地方法

```

public native void putOrderedInt(Object var1, long var2, int var4);

```

首先就要了解一下volatile的实现其实是加了内存屏障，其保证volatile变量的修改可以立刻让所有的线程可见，保证了可见性。而不加volatile变量的字段，JMM不保证普通变量的修改立刻被所有的线程可见：

1. 保证写volatile变量会强制把CPU写缓存区的数据刷新到内存
2. 读volatile变量时，使缓存失效，强制从内存中读取最新的值
3. 由于内存屏障的存在，volatile变量还能阻止重排序

但有时候我们不一定要用volatile变量来修饰共享的变量，如在使用所的情况：

1. 因为访问共享状态之前先要获得锁，Lock.lock()方法能够获得锁，而获得锁的操作和volatile变量的读操作一样，会强制使CPU缓存失效，强制从内存读取变量。（底层也是通过加内存屏障实现）
2. Lock.unlock()方法释放锁时，和写volatile变量一样，会强制刷新CPU写缓冲区，把缓存数据写到主内存

而lazySet()的用法和上面的优化是一个道理，就是在不需要让共享变量的修改立刻让其他线程可见的时候，以设置普通变量的方式来修改共享状态，可以减少不必要的内存屏障，从而提高程序执行的效率。

详情2 getAndSet方法

```

public final int getAndSet(int newValue) {
    // 调用unsafe的getAndSetInt方法
    // 参数: 1.当前实例 2.value实例变量的偏移量 3.当前value要设置的新值
    return unsafe.getAndSetInt(this, valueOffset, newValue);
}

```

其中调用了unsafe类的getAndSetInt方法

```

// var1: 要操作的对象
// var2: 该对象中整数字段的内存地址偏移量（通过Unsafe类中的objectFieldOffset方法获取）
// var3: 要设置的新值
// return: 操作前的原始数值
public final int getAndSetInt(Object var1, long var2, int var4) {
    int var5;
    do {
        // 读取var1对象中var2偏移量位置的整数值。这个操作是“volatile”读取，保证读取操作的内存可见性
        var5 = this.getIntVolatile(var1, var2);

        // 尝试将var5位置的值（即var1对象在var2位置的值）更新为var4，如果当前值等于var5，则更新成功，否则更新失败
    } while(!this.compareAndSwapInt(var1, var2, var5, var4));
    return var5;
}

```

详情3 compareAndSet方法

```

public final boolean compareAndSet(int expect, int update) {
    return unsafe.compareAndSwapInt(this, valueOffset, expect, update);
}

```

其中调用了unsafe类的compareAndSet方法，同样其是一个本地方法

```

public final native boolean compareAndSwapInt(Object var1, long var2, int var4,
int var5);

```

- AtomicBoolean
- AtomicLong

2. 原子更新数组类型

3. 原子更新引用类型

4. 原子更新字段类型

三、JUC锁

3.1 LockSupport

参考文章: <https://pdai.tech/md/java/thread/java-thread-x-lock-LockSupport.html>

3.2 锁核心类AQS

太过于抽象了, 要对AQS学习, 先从其各个具体实现学习。

1. 相关文章

- 关于各个状态的说明: <https://www.cnblogs.com/yanlong300/p/10953185.html>

2. AQS简介

AQS (AbstractQueuedSynchronizer) , 一个用来构建锁和同步器的框架, 使用AQS能简单且高效地构造出应用广泛的大量的同步器, 比如我们提到的ReentrantLock, Semaphore, 其他的诸如ReentrantReadWriteLock, SynchronousQueue, FutureTask等等皆是基于AQS的。当然, 我们自己也能利用AQS非常轻松容易地构造出符合我们自己需求的同步器。